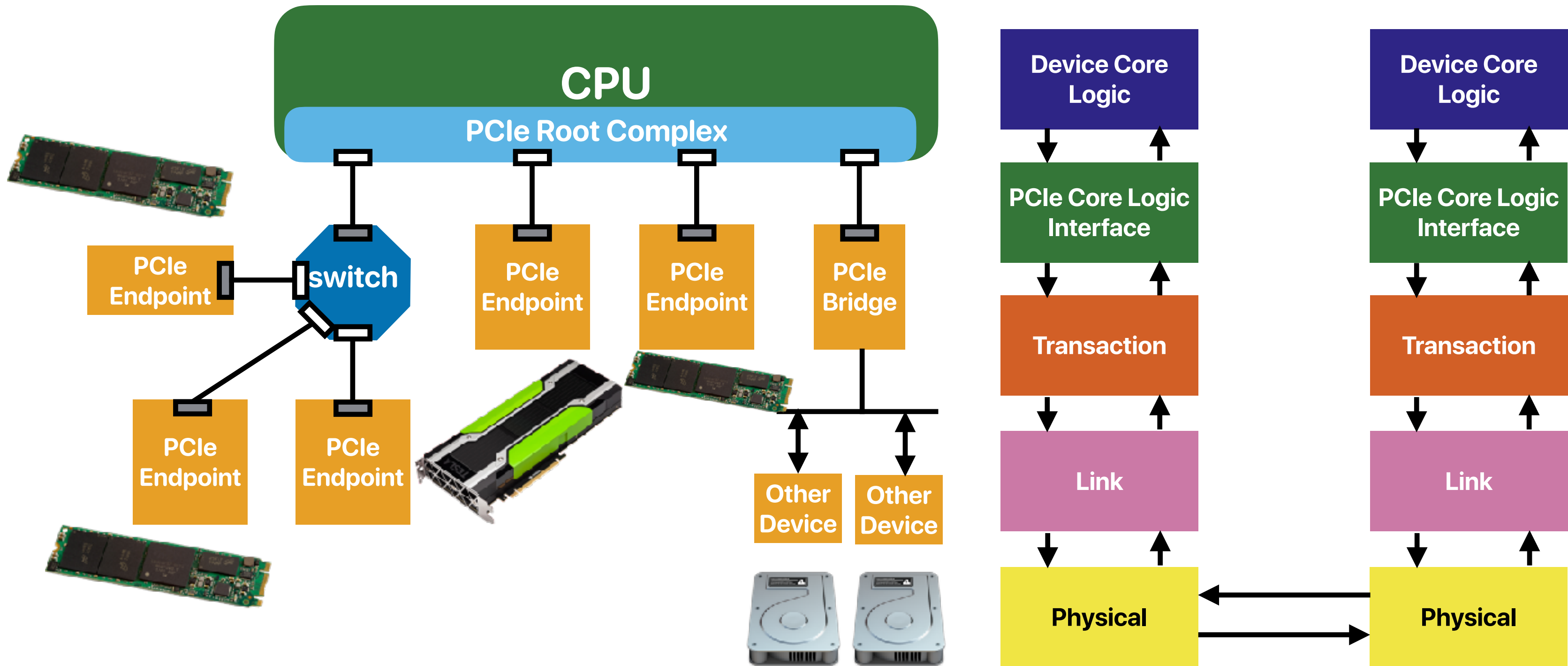


Basics on Storage Systems/ Devices

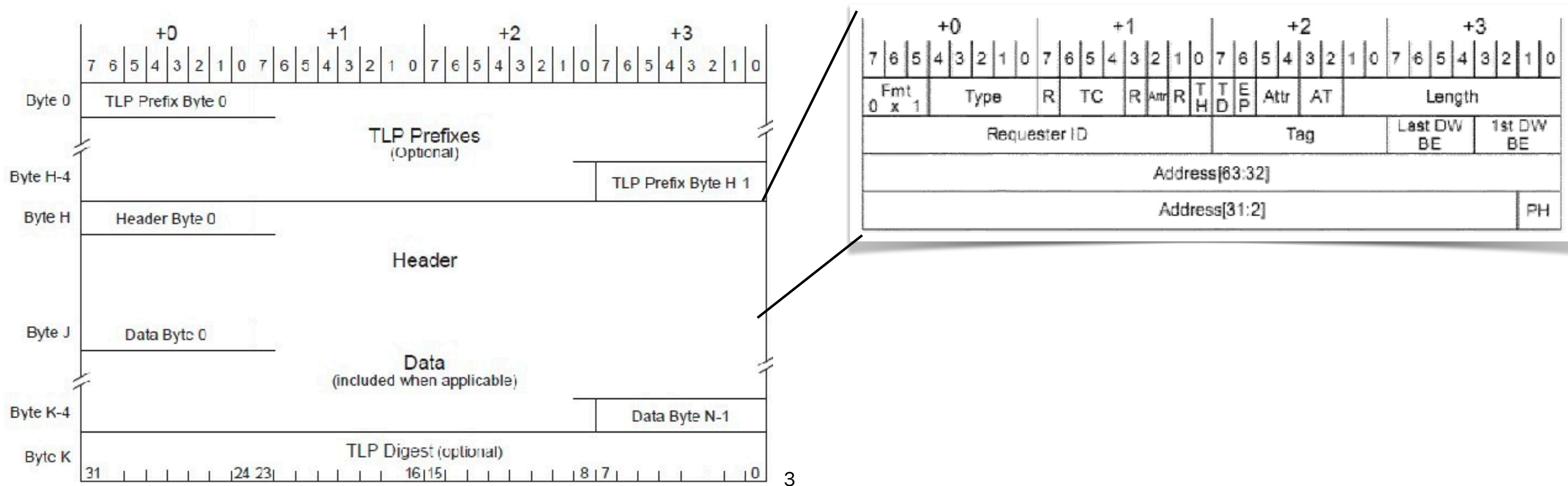
Hung-Wei Tseng

Recap: PCIe "Interconnect"

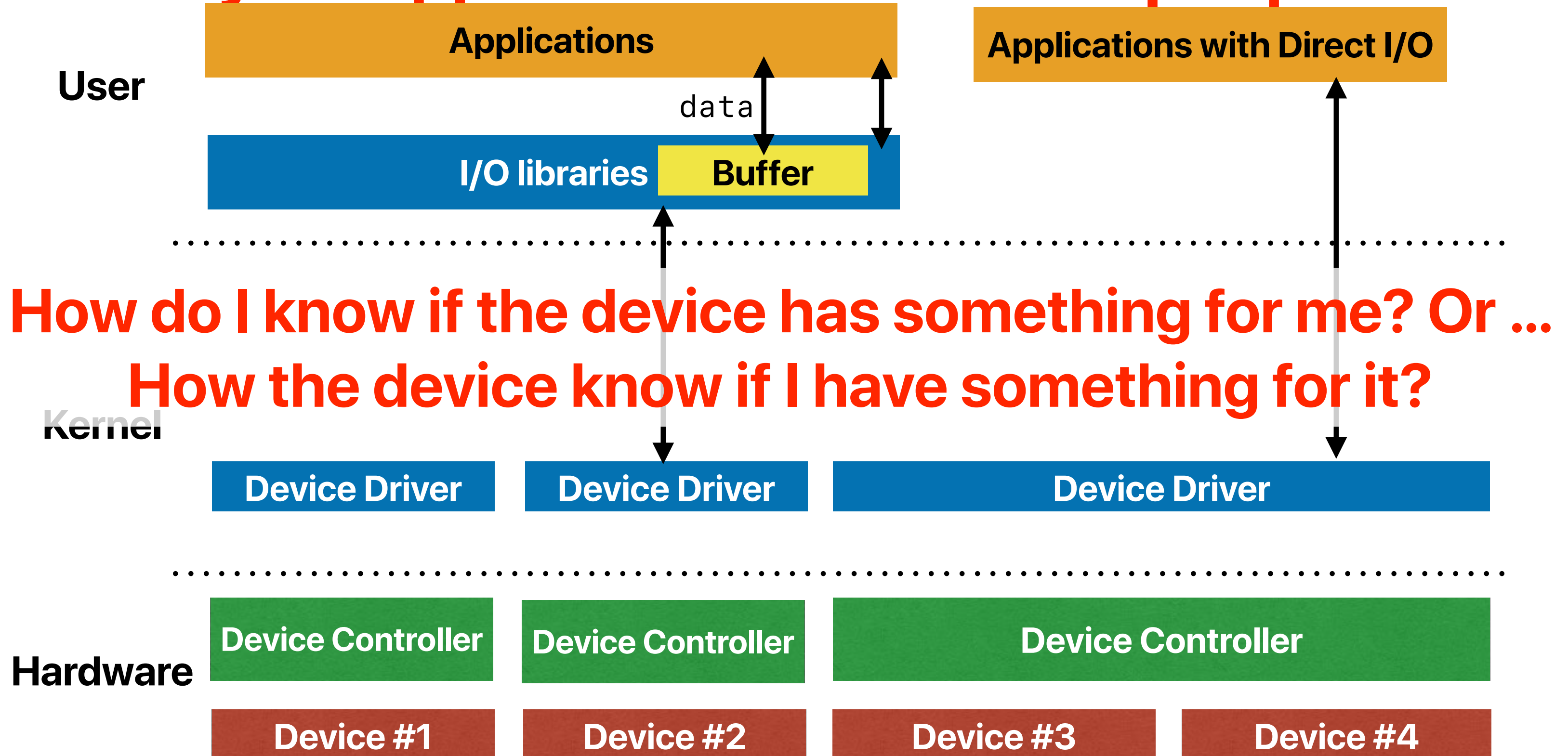


PCIe interconnect

- Very similar to computer networks
- Use "memory addresses" as the identifier for routing

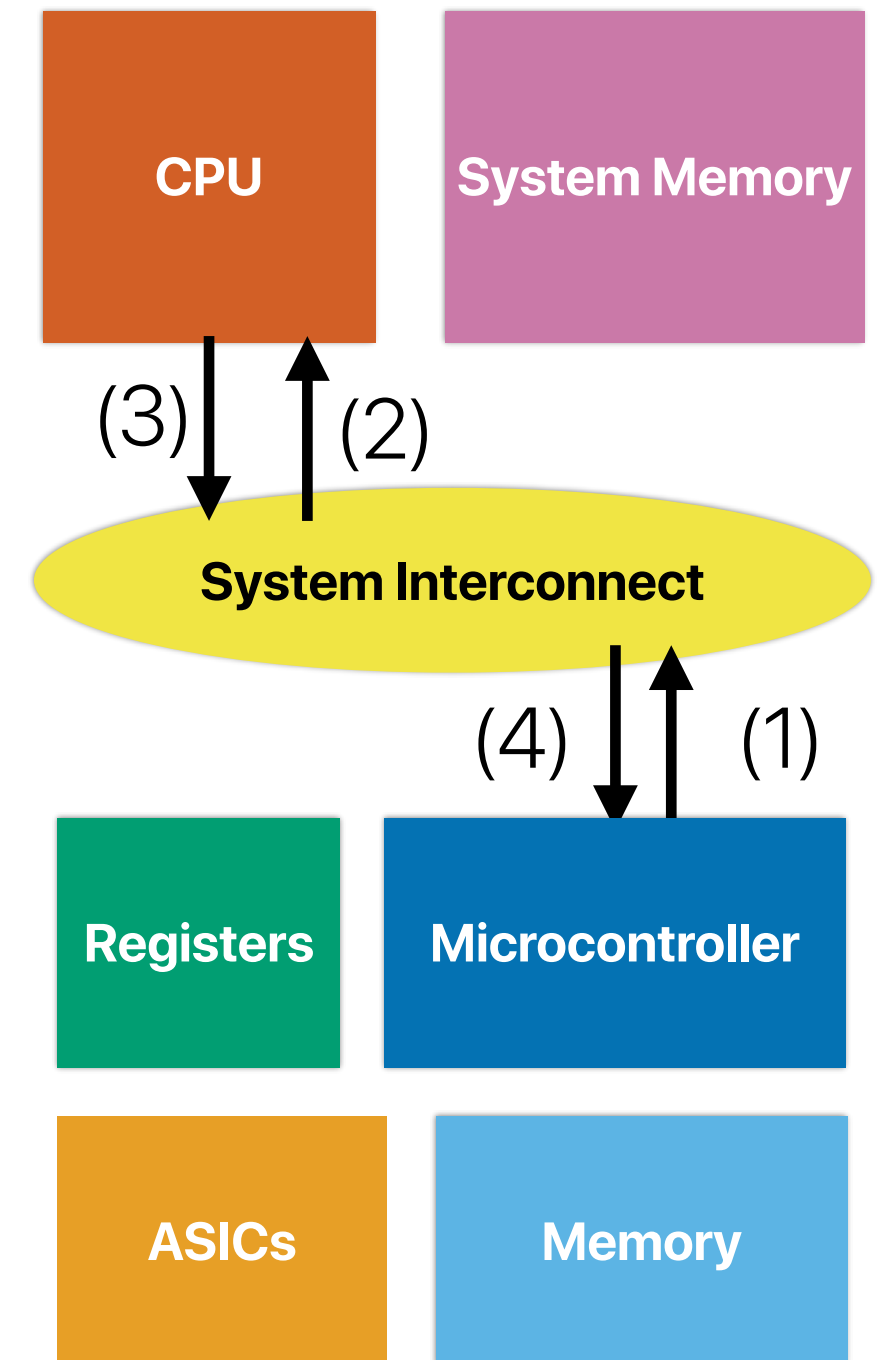


How your application interact with peripherals



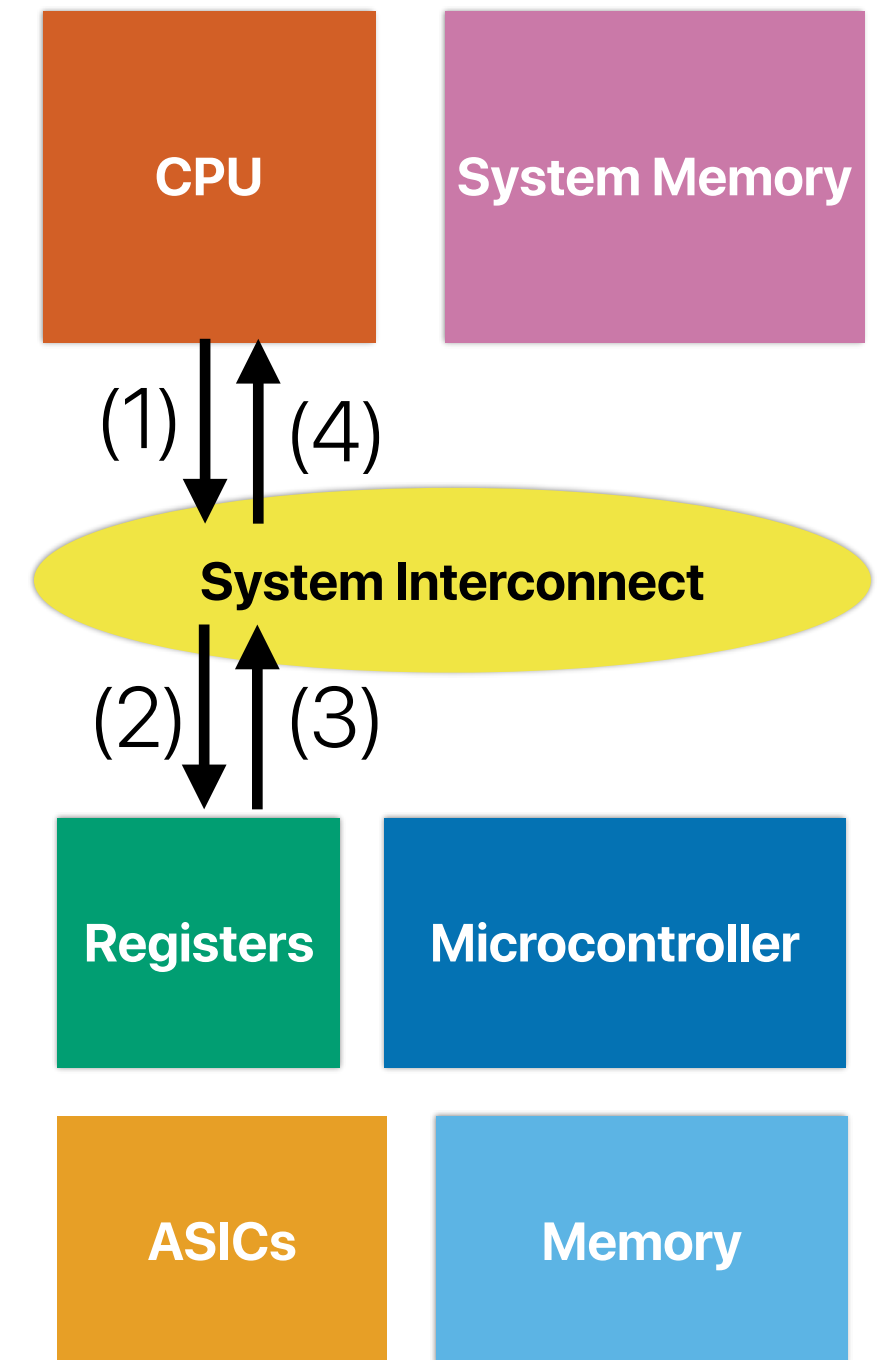
Interrupt

- The device signals the processor only when the device requires the processor/OS handle some tasks/data
- The processor only signals the device when necessary



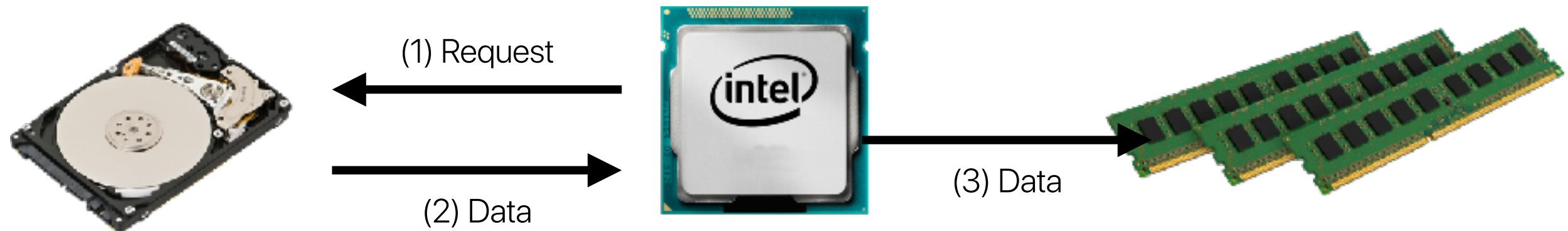
Polling

- The processor/OS constantly asks if the device (e.g. examine the status register of the device) is ready to or requires the processor/OS handle some tasks/data
- The OS/processor executes corresponding handler if the device can handle demand tasks/data or has tasks/data ready



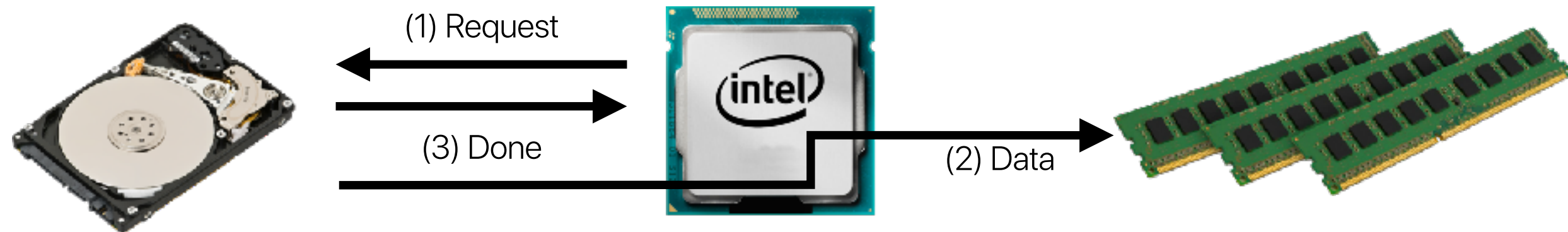
Programmed I/O (PIO)

- The processor/OS executes code and issues commands to handle the data movements between the peripheral and the main memory
 - For example, the in/out instruction in x86



Direct Memory Access (DMA)

- The OS allocates a buffer to exchange data with the peripheral
- The peripheral device can directly access the allocated buffer



Outline

- When should we poll or interrupt
- Storage devices

**What're the benefits of interrupt
compared to polling?**

Polling/Interrupt/DMA/PIO?

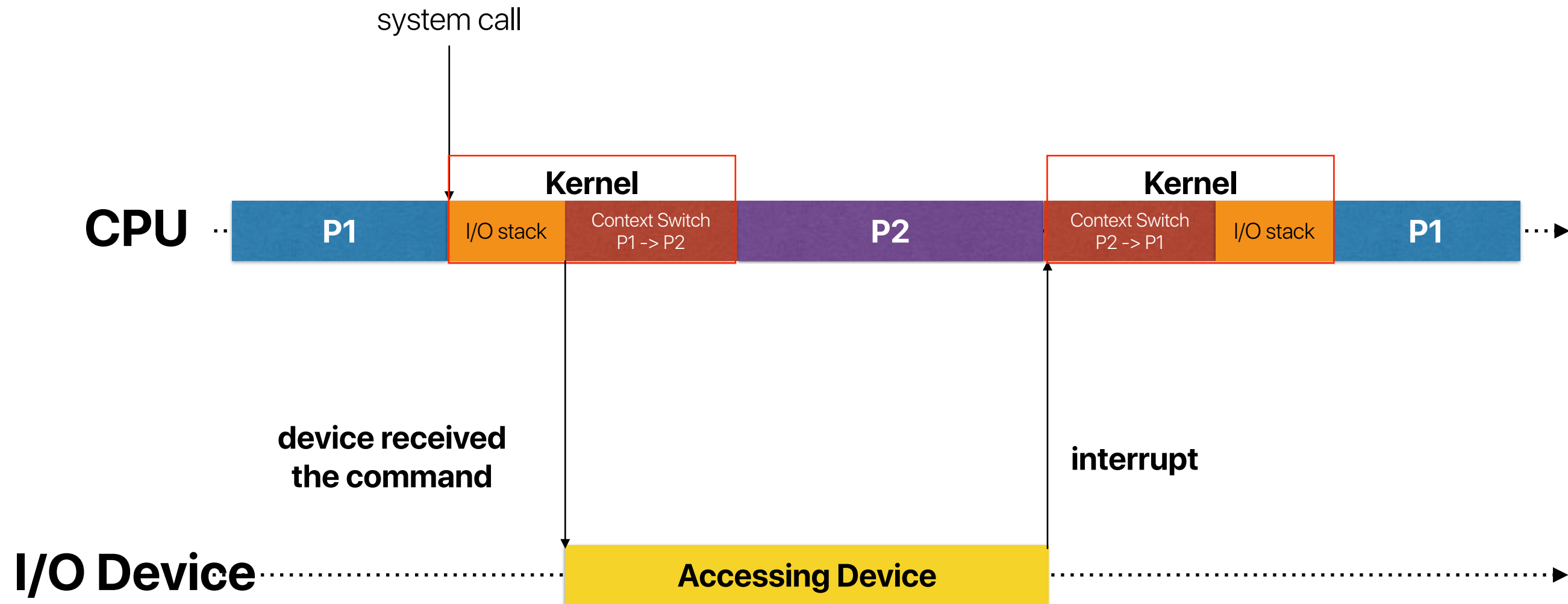
	Polling	Interrupt
DMA	CPU OPs more than IRQ Shorter latency High throughput	Lowest CPU Operations Longer latency High throughput
PIO	Most CPU operations Shorter latency Low throughput	CPU OPs more than IRQ Longer latency Low throughput

What happened during a context switch?

Recap: What happens during context switch

- Load architectural states from process control block (somewhere in the main memory, potentially a cache miss, TLB miss) — takes several microseconds if everything is in the physical memory
- Set processor registers according to the loaded architectural states
 - **Set the CR3 (page table base register in x86) register to identify the root page table node in the hierarchical page table**
 - Set the RIP (program counter in x86) to the previous execution
- **Restore virtual memory address**
 - **You must load the root page table node to the main memory at least.**
 - **TLB flush**
 - Invalidate all entries in the TLB
 - Most TLBs are not tagged, so you've to do this
- You DO NOT have to load every page content back from disk — remember that we have demand paging!

To switch or not to switch that's the question.

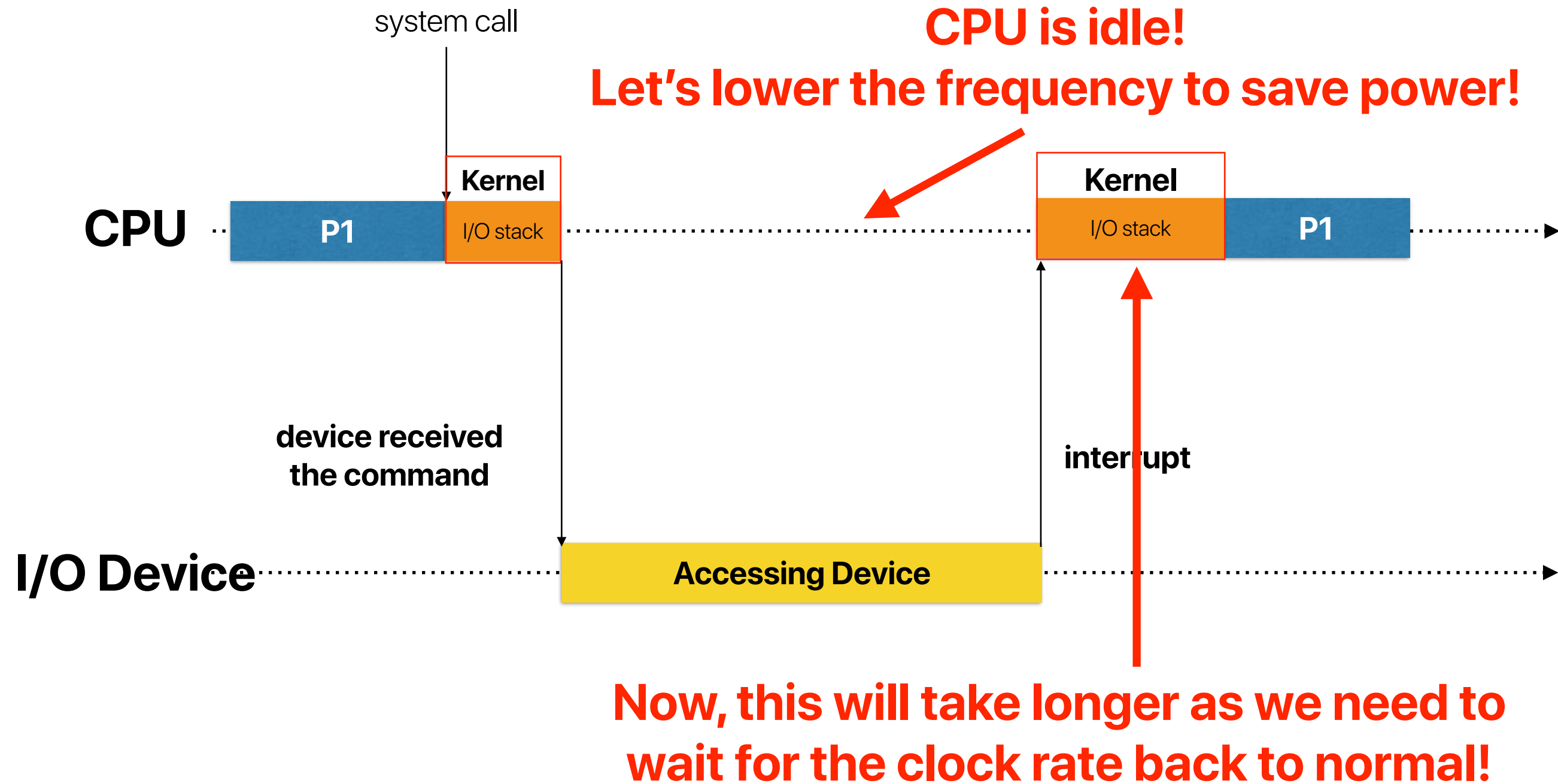


If $T_{\text{Context switch P1} \rightarrow \text{P2}} + T_{\text{Context switch P2} \rightarrow \text{P1}} < T_{\text{Accessing peripherals}}$
makes sense to context switch

But context switch overhead is not the only thing

- Cache warm up cost when you switch back
- TLB warm up cost

What if we don't switch?



Polling/Interrupt/DMA/PIO?

	Polling	Interrupt
DMA	CPU OPs more than IRQ Shorter latency High throughput	Lowest CPU Operations Longer latency High throughput
PIO	Most CPU operations Shorter latency Low throughput	CPU OPs more than IRQ Longer latency Low throughput

When should we poll? When should we interrupt

- Interrupt is only a good option if the benefit from context switching or energy saving is larger than waiting for the I/O to finish
- In general, applying polling on faster devices
 - DRAM
 - Non-volatile memory (e.g., flash, PCM)

Hard Disk Drives

Numbering the disk space with block addresses

Disk blocks

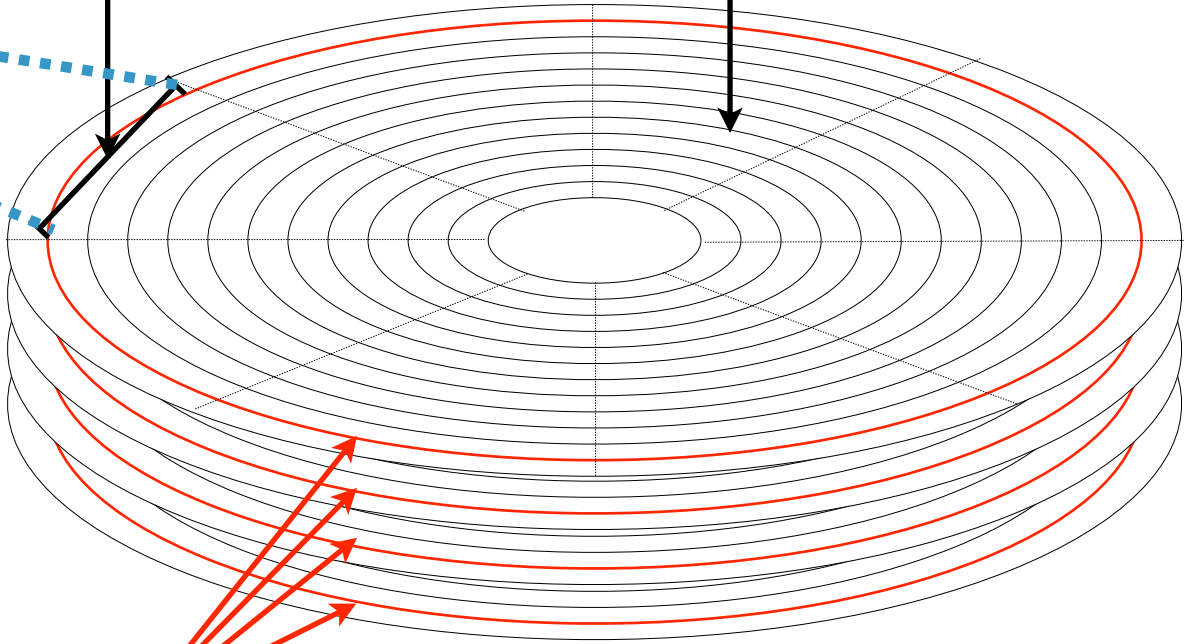
0								7
8								15
16								23
24								31
32								39
40								47
48								55
56								63

...

Rotational delay — rotate the disk to a specific angle
Seek time — position the disk head to a specific track

sector

track



cylinder

SATA Standard

	Speed	
SATA Express	16 Gbit/s	1.97 GB/s
SATA revision 3.0	6 Gbit/s	600 MB/s
SATA revision 2.0	3 Gbit/s	300 MB/s
SATA revision 1.0	1.5 Gbit/s	150 MB/s



Seagate Barracuda 12

- SATA 3.0 (600MB/s in theory), 7200 R.P.M., seek time around 8 ms. Assume the controller overhead is 0.2ms. What's the **latency** and **bandwidth** of accessing a **512B** sector?

Latency = seek time + rotational delay + transfer time + controller overhead

$$\begin{aligned} & 8 \text{ ms} + \frac{1}{2} \times \frac{1}{\frac{7200}{60}} + \frac{\frac{0.5}{1024}}{600} + 0.2 \text{ ms} \\ &= 8 \text{ ms} + 4.17 \text{ ms} + 0.000835 \text{ us} + 0.2 \text{ ms} = 12.36 \text{ ms} \end{aligned}$$

Bandwidth = volume_of_data over period_of_time

$$= \frac{0.5KB}{12.36ms} = 40.45KB/sec$$

Seagate Barracuda 12

- SATA 3.0 (600MB/s in theory), 7200 R.P.M., seek time around 8 ms. Assume the controller overhead is 0.2ms. What's the **latency** and **bandwidth** of accessing consecutive **4KB** data?

Latency = seek time + rotational delay + transfer time + controller overhead

$$8 \text{ ms} + \frac{1}{2} \times \frac{1}{\frac{7200}{60}} + \frac{\frac{4}{1024}}{600} + 0.2 \text{ ms}$$

$$= 8 \text{ ms} + 4.17 \text{ ms} + 0.000651041667 \mu\text{s} + 0.2 \text{ ms} = 12.37 \text{ ms}$$

Bandwidth = volume_of_data over period_of_time

$$= \frac{4KB}{25.69ms} = 0.315 \text{ MB/sec} \quad \text{Trading latencies with bandwidth}$$

Seagate Barracuda 12

- SATA 3.0 (600MB/s in theory), 7200 R.P.M., seek time around 8 ms. Assume the controller overhead is 0.2ms. What's the **latency** and **bandwidth** of accessing consecutive **4MB** data?

Latency = seek time + rotational delay + transfer time + controller overhead

$$8 \text{ ms} + \frac{1}{2} \times \frac{1}{\frac{7200}{60}} + \frac{4}{600} + 0.2 \text{ ms}$$

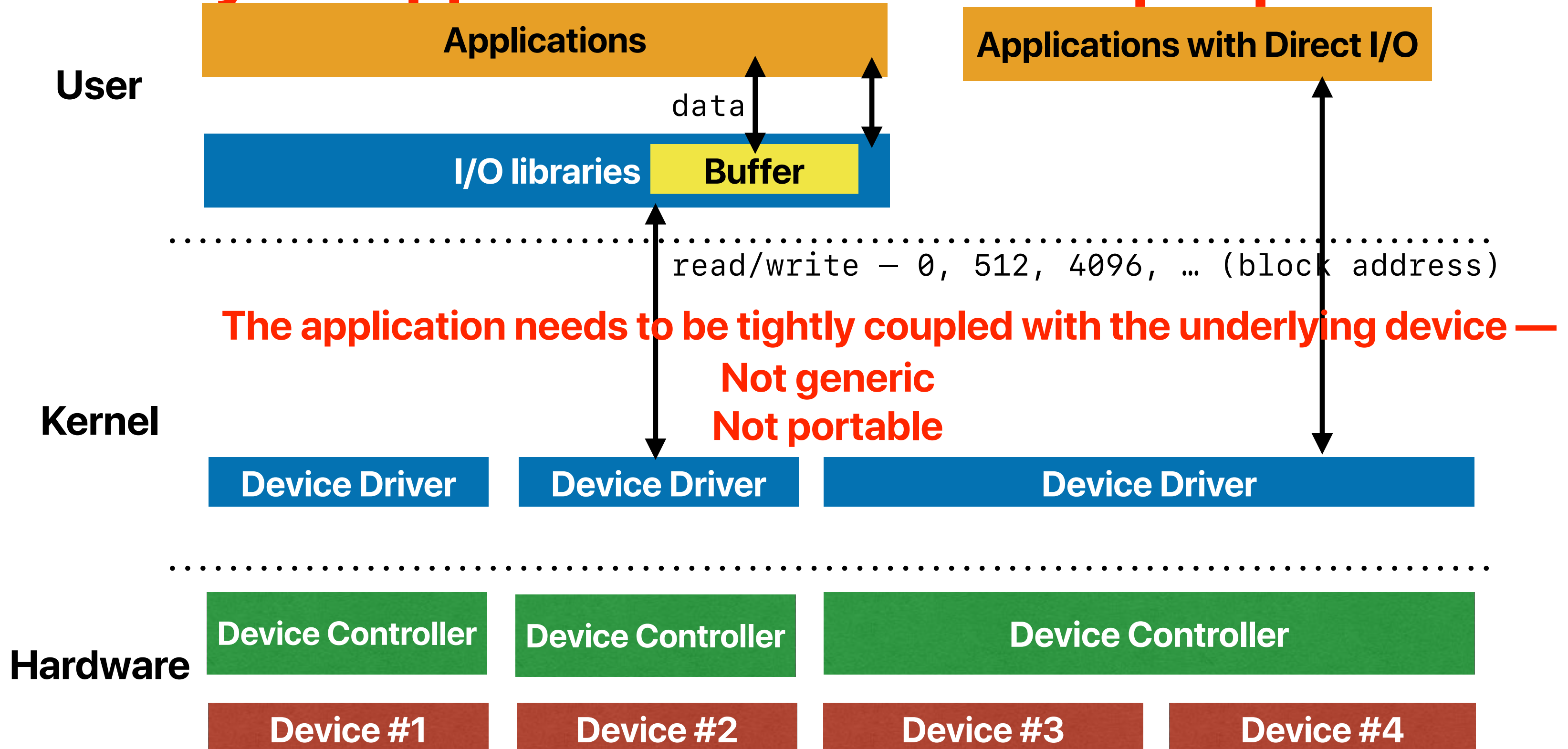
$$= 8 \text{ ms} + 4.17 \text{ ms} + 6.67 \text{ ms} + 0.2 \text{ ms} = 19.04 \text{ ms}$$

Bandwidth = volume_of_data over period_of_time

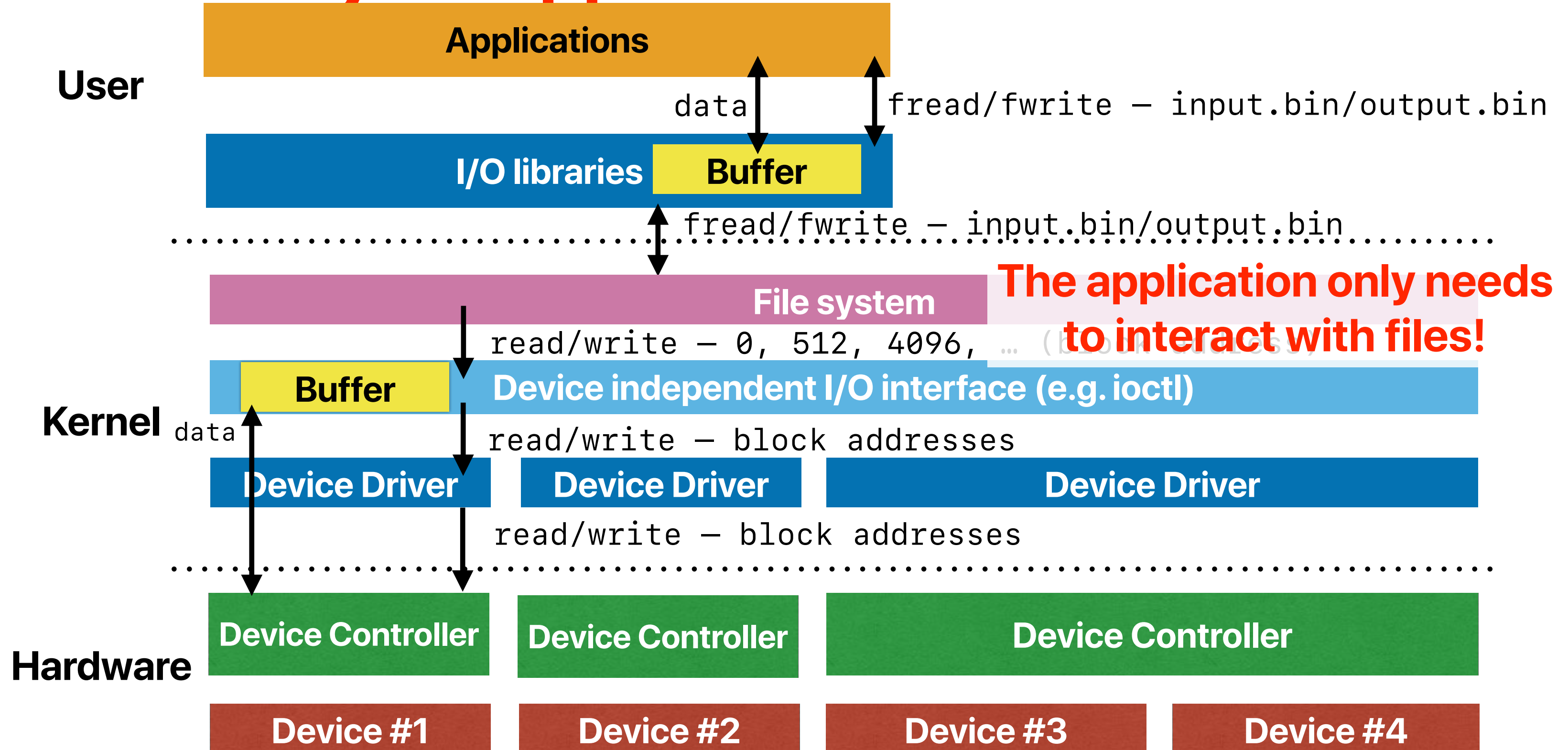
$$= \frac{4MB}{19.04ms} = 210.1MB/sec$$

Trading latencies with bandwidth

How your application interact with peripherals



How your application reaches H.D.D.

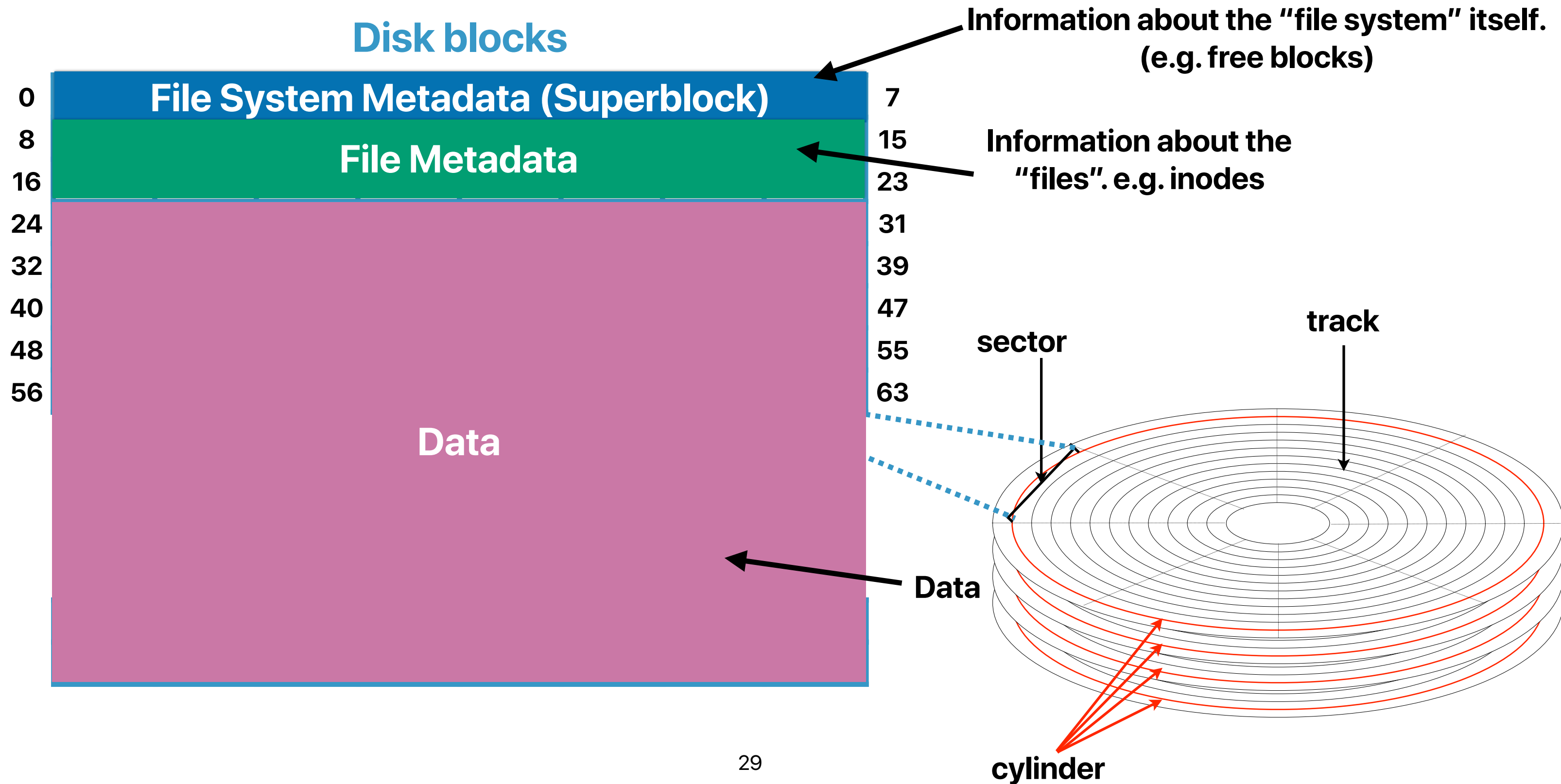


What would good practices for file systems on hard disk drives?

Design for a good file system for HDDs

- Should avoid lots of seeks (different tracks) when accessing a file — **consecutive storage for the same file**
- Optimize bandwidth utilization — **consecutive storage for the same file**
- Support arbitrary file size
- Crash recovery

How the original UNIX file system use disk blocks



How to reach /home/hungwei/CS202/foo.c

Superblock

Disk blocks

File System Metadata (Superblock)

File Metadata

index node (inode)

inode 1	
owner id	0
permission	755
type	dir
address	24
...	

inode 15	
owner id	0
permission	755
type	dir
address	31
...	

inode 21	
owner id	0
permission	755
type	dir
address	34
...	

home	
tvler	20
hungwei	21

#include	
<stdio.h>	
.	
.	
.	
.	

CS202	
bar.c	18
foo.c	19

inode 16	
owner id	0
permission	755
type	dir
address	44
...	

/	
usr	13
var	14
home	15

hungwei	
CS202	16
Dropbox	17

inode 19	
owner id	0
permission	755
type	file
address	55
...	

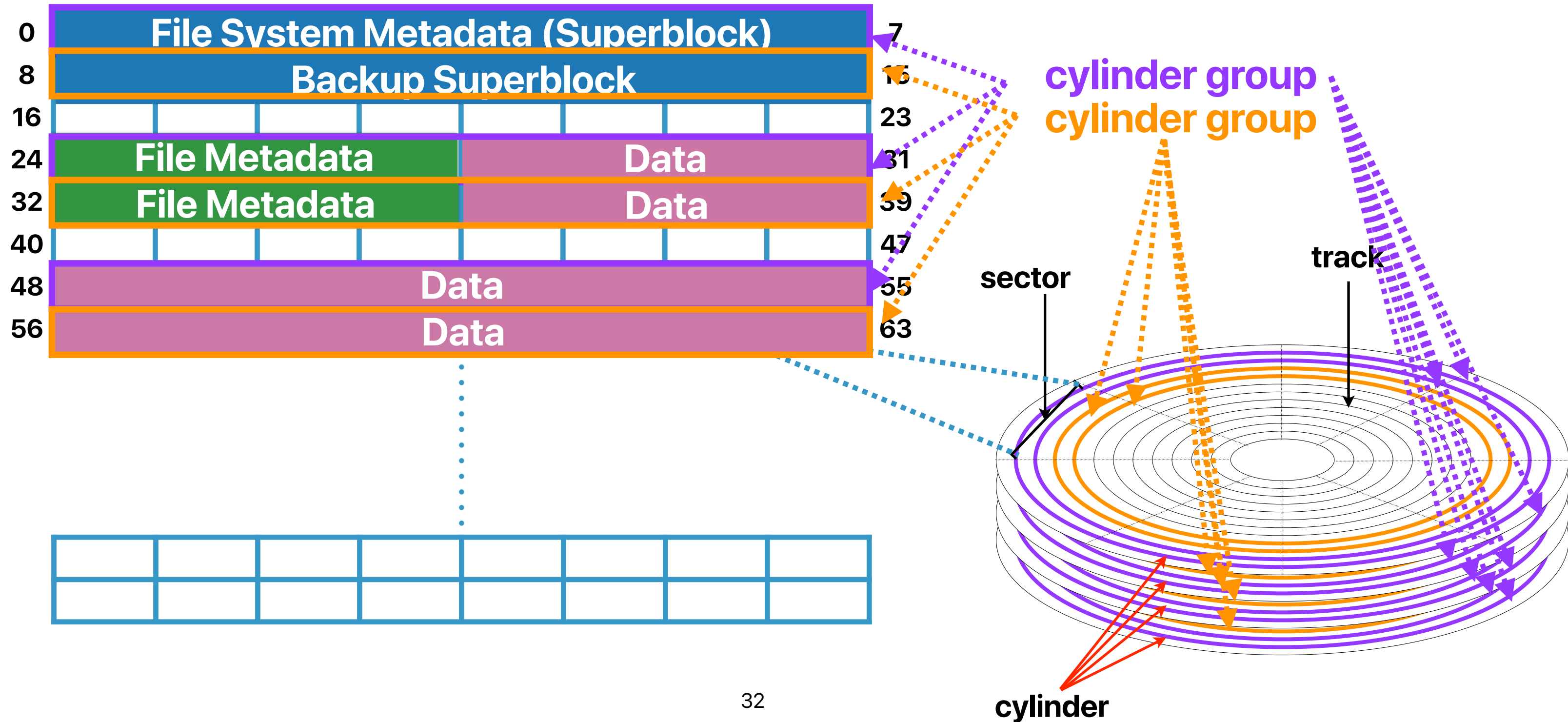
0	File System Metadata (Superblock)						
8							
16							
24							
32							
40							
48							
56							

Design for a good file system for HDDs

- Should avoid lots of seeks (different tracks) when accessing a file
 - inodes are separated from data locations
 - data blocks belong to the same file can be spread out
- Low bandwidth utilization
 - only the very last is retrieving data
 - 1 out 11 in our previous example — less than 10% if files are small
- Limited file size
- Crash recovery

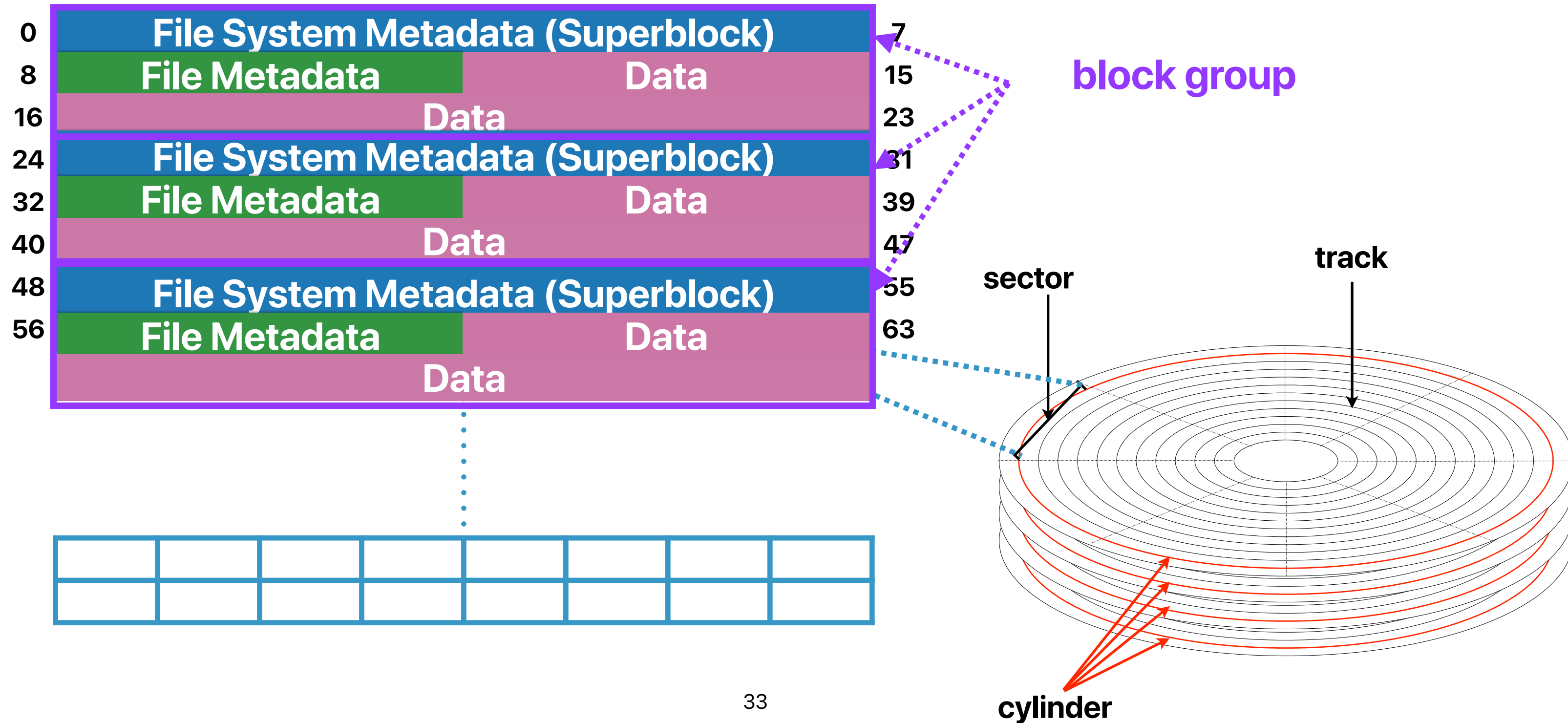
How FFS use disk blocks

Disk blocks



How ExtFS use disk blocks

Disk blocks

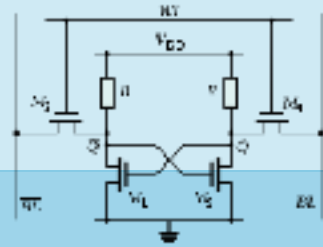


Memory technologies we have today

Volatile Memory

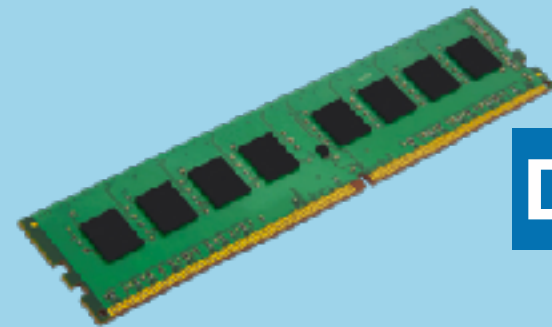
Non-Volatile Memory

100ps



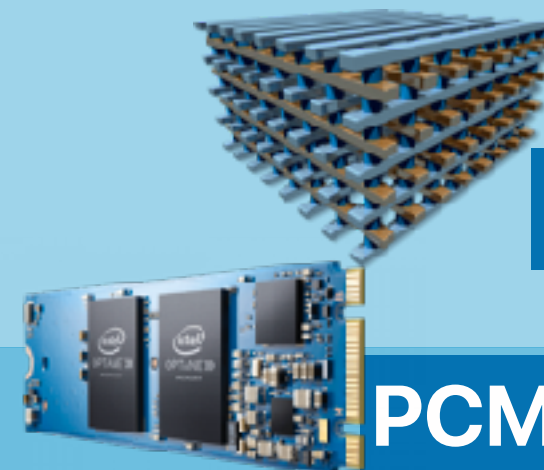
SRAM

ns



DRAM

us

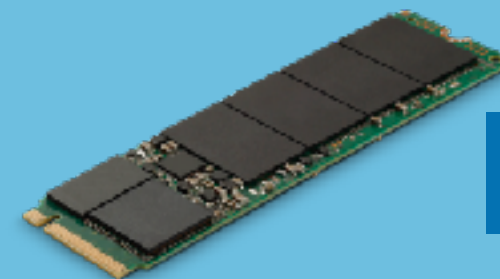


RRAM

PCM

3DXPoint

ms



Flash memory

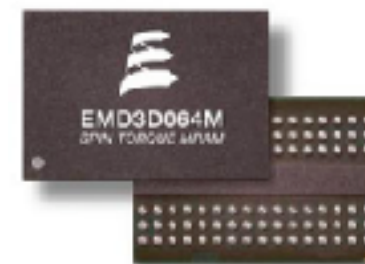


Hard Disk Drives

Based on what you've read, what are the differences between magnetic hard disks & NAND flash?

NAND Flash v.s. hard disks

Non-volatile memory technologies



	H.D.D	Flash	Optane	STT-MRAM
Latency	~ 10-15 ms	~ 100 us (read) ~ 1 ms (write)	7 us (read) 18 us (write)	35 ns
Bandwidth	~200 MB/Sec	3.5 GB/sec (read) 2.1 GB/sec (write)	1.35 GB/sec (read) 290 MB/sec (write)	
Dollar/GB	0.0295	0.583	2.18	

Flash is still the most convincing technology for now

NAND Flash v.s. hard disks

		NAND	Hard Disks
granularities	read	page	sector
	write	page	sector
	erase	block (a group of pages)	sector
performance	read	us	ms
	write	100s of us — ms	ms
	erase	ms	ms
program		1,000—100,000 times	unlimited

NAND flash is just odd!

- Modern SSDs are based on NAND flash memory
- Different operation granularities
 - Read/Program in pages
 - Erase in blocks (64-384 pages)
- Performance of operation varies
 - Read — tens of us
 - Program — hundreds of us
 - Erase — ms
- Limited erase cycles
 - Only 1000 times for "QLC"

Electrical Computer Science Engineering

277

つくづく

